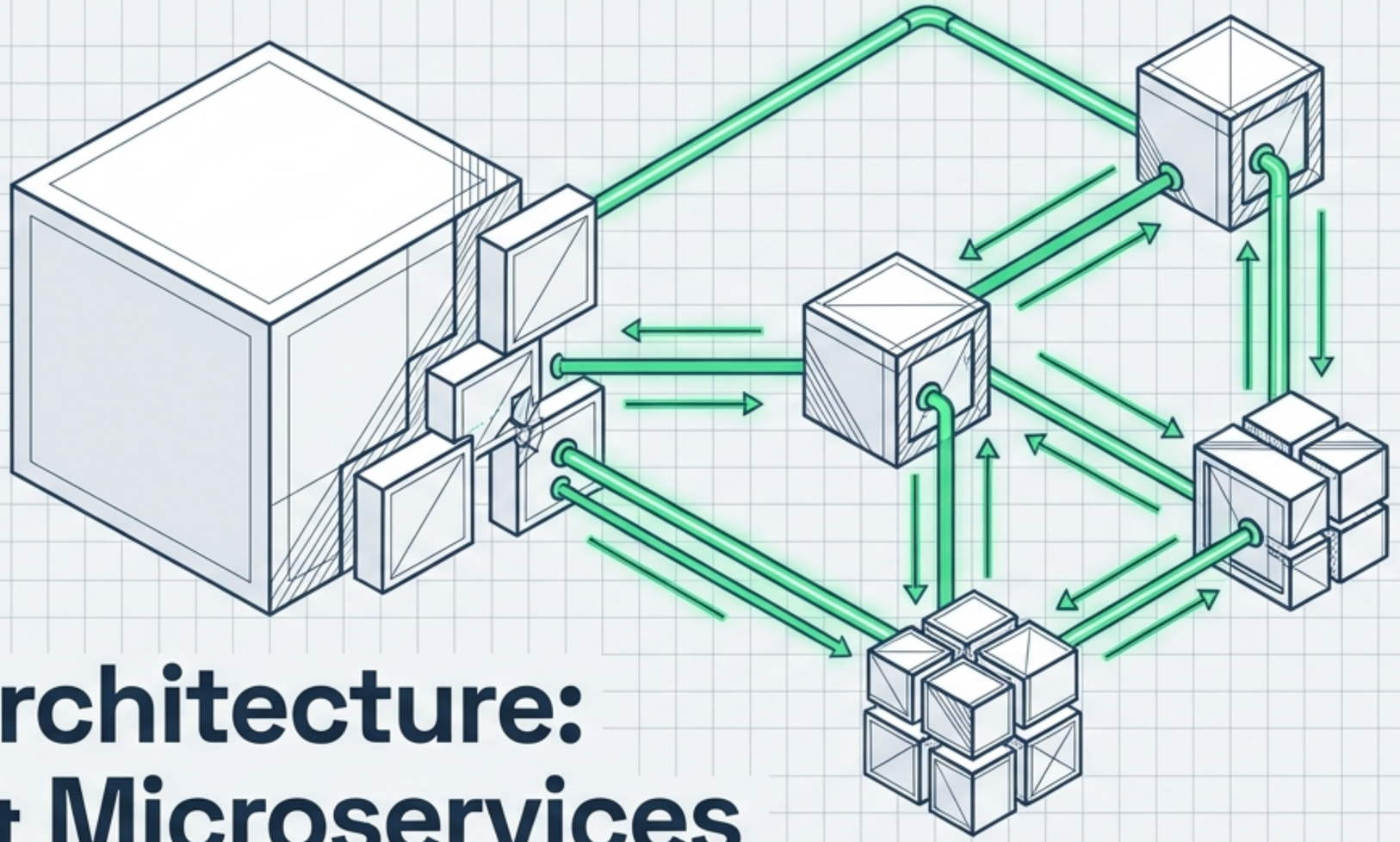
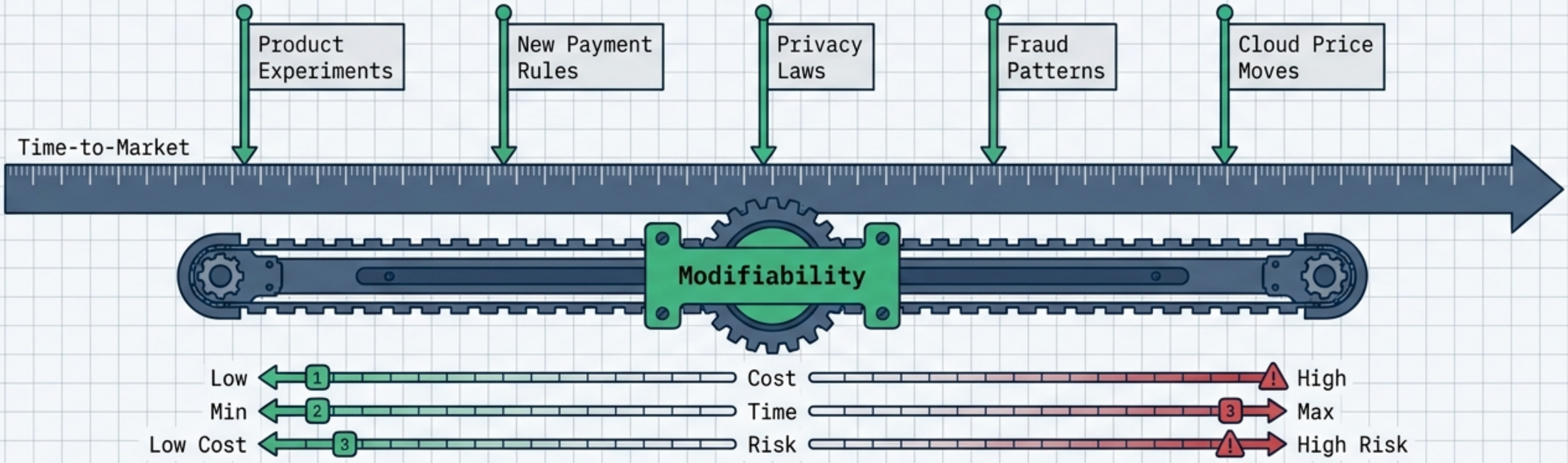


Software Architecture: Flexibility & Microservices

Navigating the trade-offs of system evolution, operational maturity, and distributed data.

Architecture either pays or taxes every change the business demands.



Flexibility / Modifiability

The ease of evolving the system without excessive cost or risk. Investors and regulators care about time-to-market; architecture dictates this speed.

The Reality of Churn

External integrations churn far more frequently than internal sorting algorithms. Architectural boundaries must match these varying rates of change.

Microservices do not remove coupling; they move it to network contracts and data consistency stories.

Cohesion

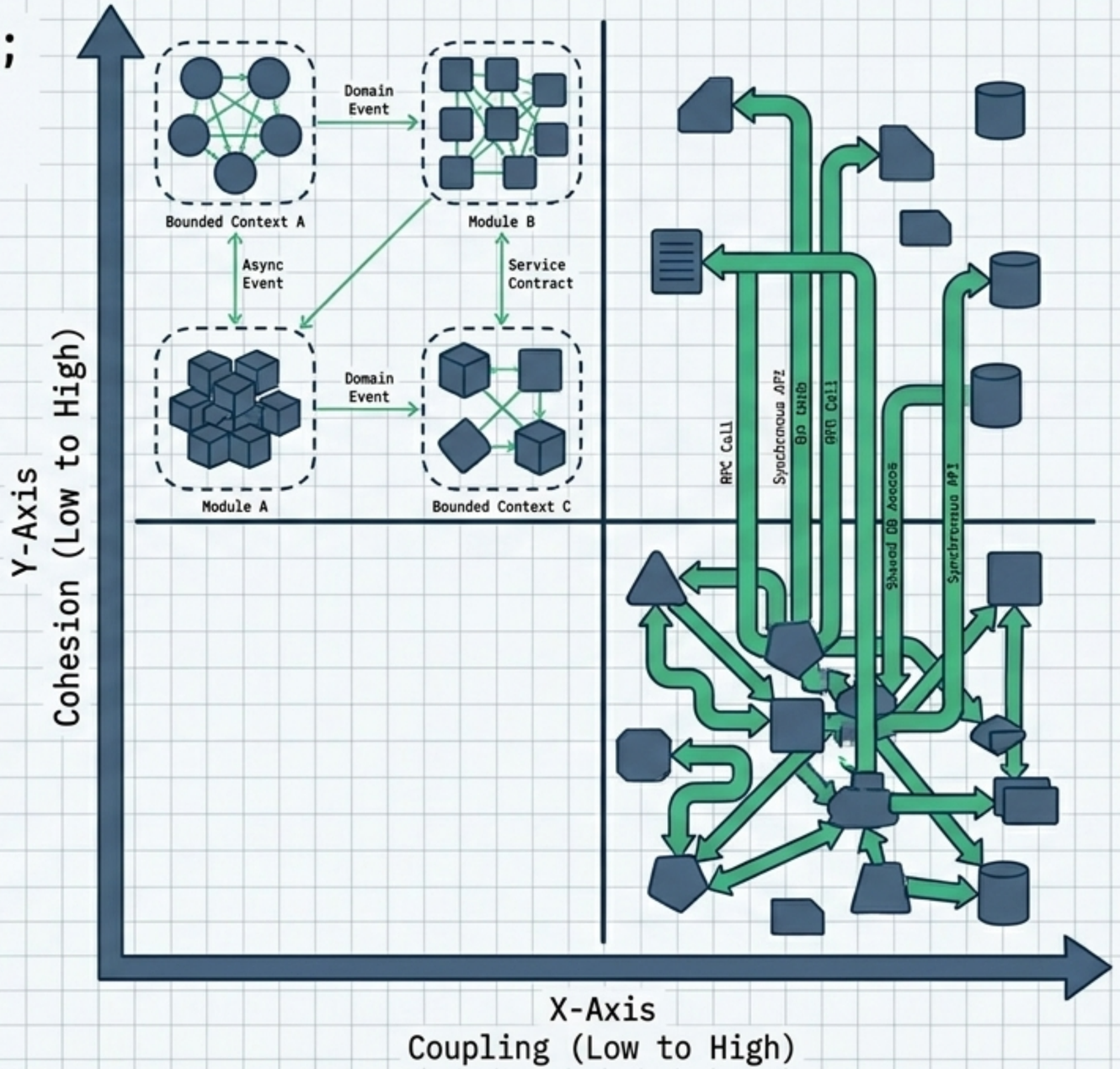
Things that change together, stay together. The degree to which elements inside a module belong together.

Coupling

The degree of interdependence. How much one module forces another to change or deploy.

The Insight

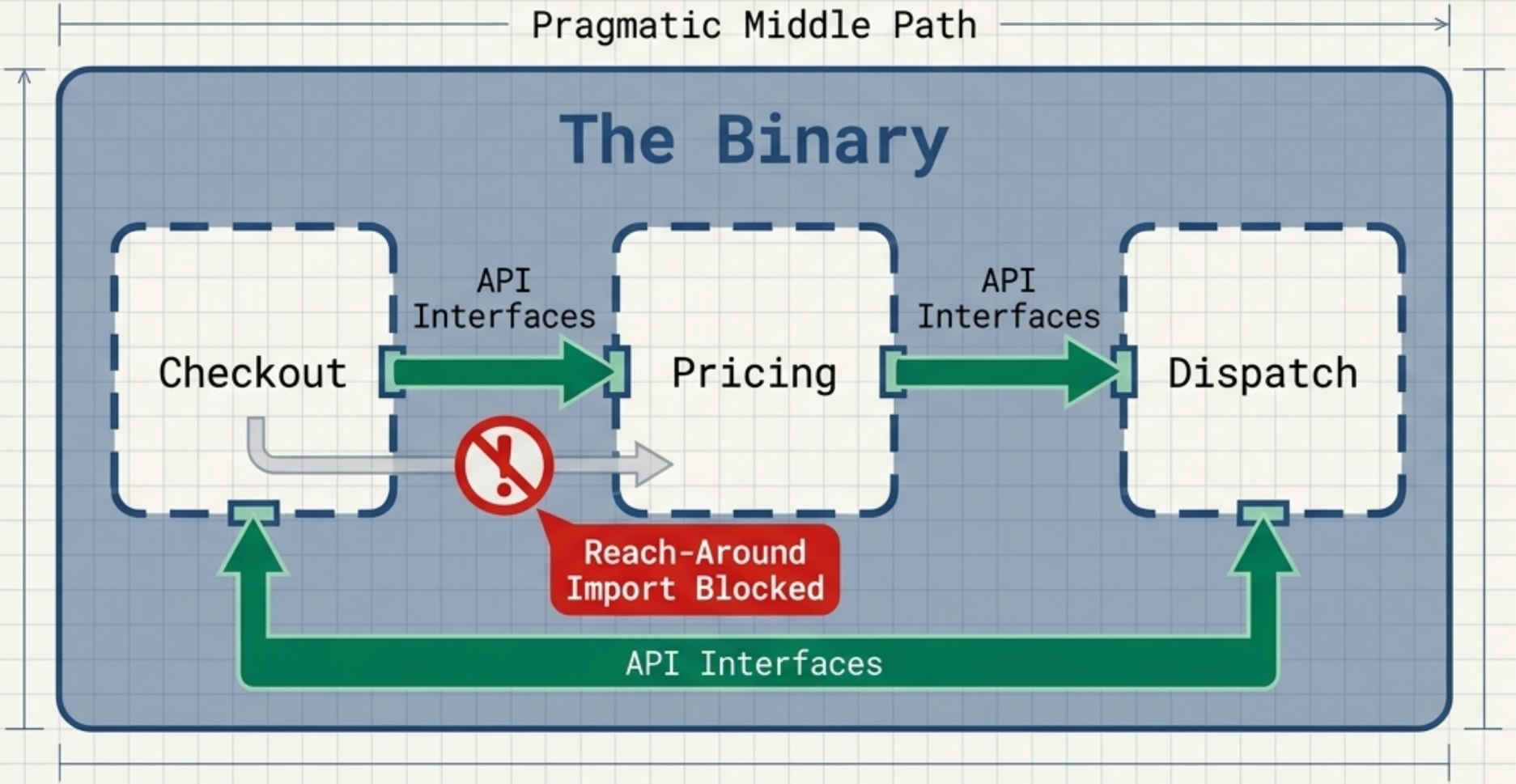
Splitting a highly coupled system into microservices just turns in-memory coupling into fragile network coupling.



Evaluate architectural topologies based on your team scale and operational maturity.

	Monolith	Modular Monolith	Microservices	Distributed Monolith
Deployment	One primary deployable	One binary, strict internal seams	Autonomous deployments	Many processes, one release train
Data Strategy	ACID transactions	Single database	Database per service	Shared DB / Synchronous rings
Team Scale	Small teams / unclear domains	Medium teams	Highly scaled, parallel teams	Coupled teams
Primary Risk	Loss of velocity as code/teams scale	Slipping discipline, spaghetti code	Exploding operational complexity	High network faults without independent velocity ⚠

Practice bounded contexts inside a single binary before paying the remote procedure call tax.



The Stepping Stone

Not every product needs **microservices** on day one.

A monolithic architecture allows simple grep-and-replace refactoring and easy ACID transactions.

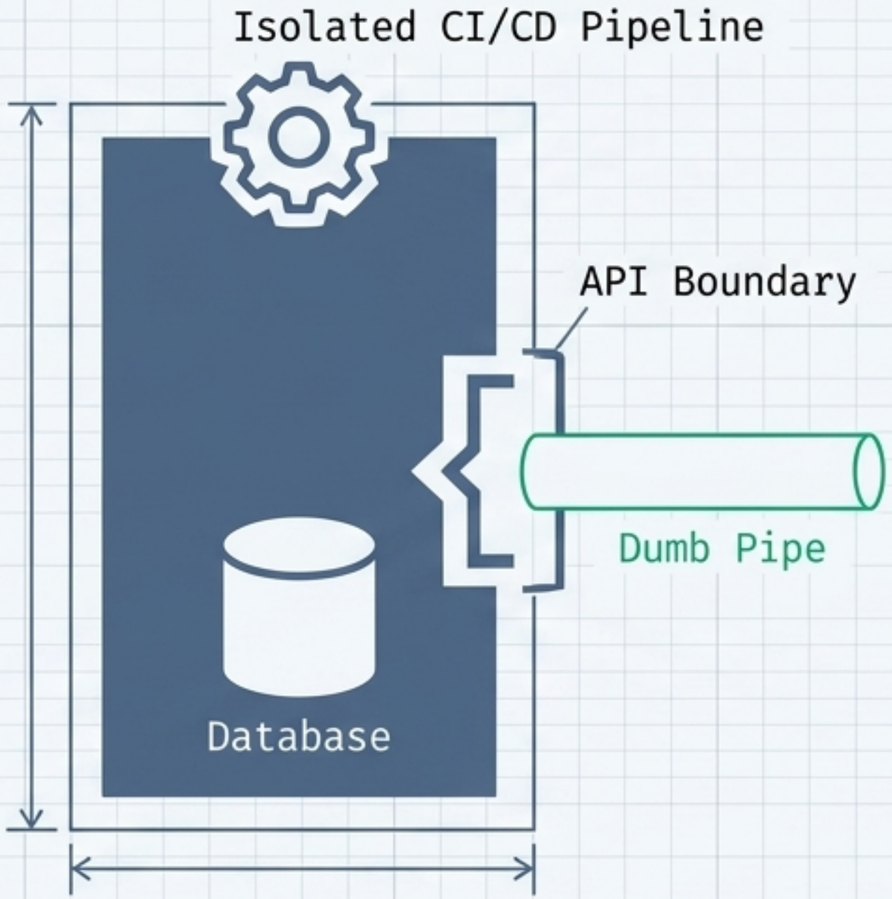
Module Boundaries

Enforce seams via packages and build targets.

If internal modules fight, splitting them into separate network processes will not magically create autonomy.

Service boundaries align with business capabilities, not technical layers or lines of code.

True Microservice



Smart Endpoints, Dumb Pipes

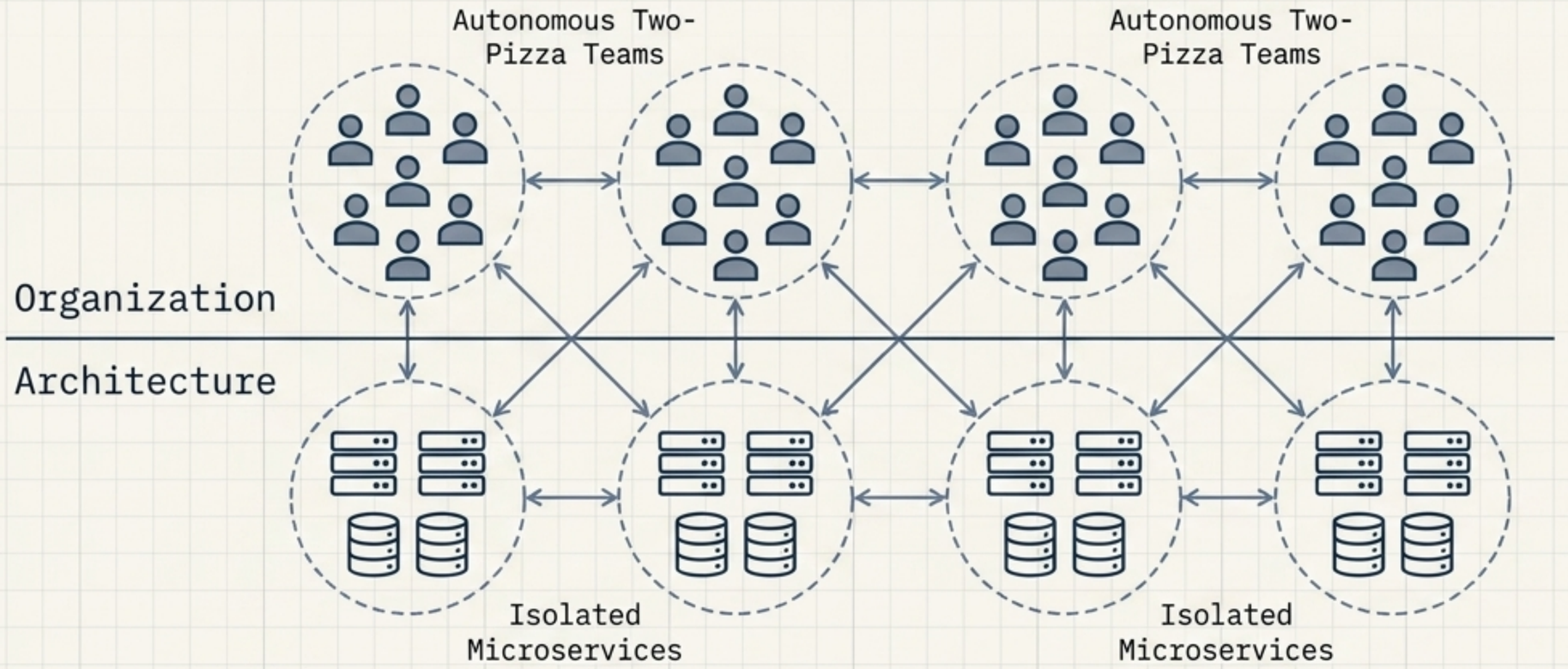
Business logic lives inside the service; infrastructure messaging stays simple. Avoid overloading middleware.

Size = Team Ownership + Clear Context + Deploy Cadence

⚠ Nano-Service Warning

Tiny-for-tiny's-sake explodes operational costs. If two services always deploy together, merge them until the boundary is real.

You must structure teams as autonomous units to achieve architectural autonomy.



Conway's Law

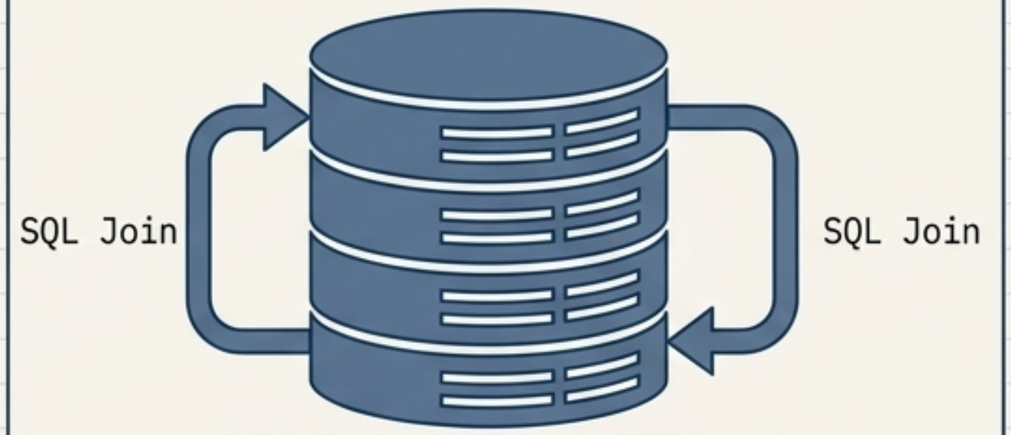
Organizations ship architectures that copy their communication lines.

The Reverse Conway Maneuver

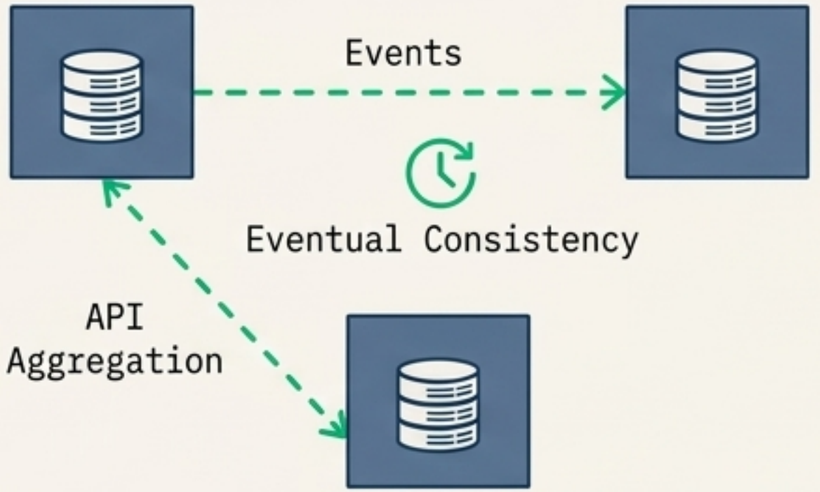
Intentionally reshape your teams to encourage your desired architecture. Microservices without team boundaries become solo-maintained, tightly coupled sprawl.

Independent schema evolution requires abandoning shared row-level databases.

Simple ACID Transactions



The Distributed Reality



The Rule: Database per service

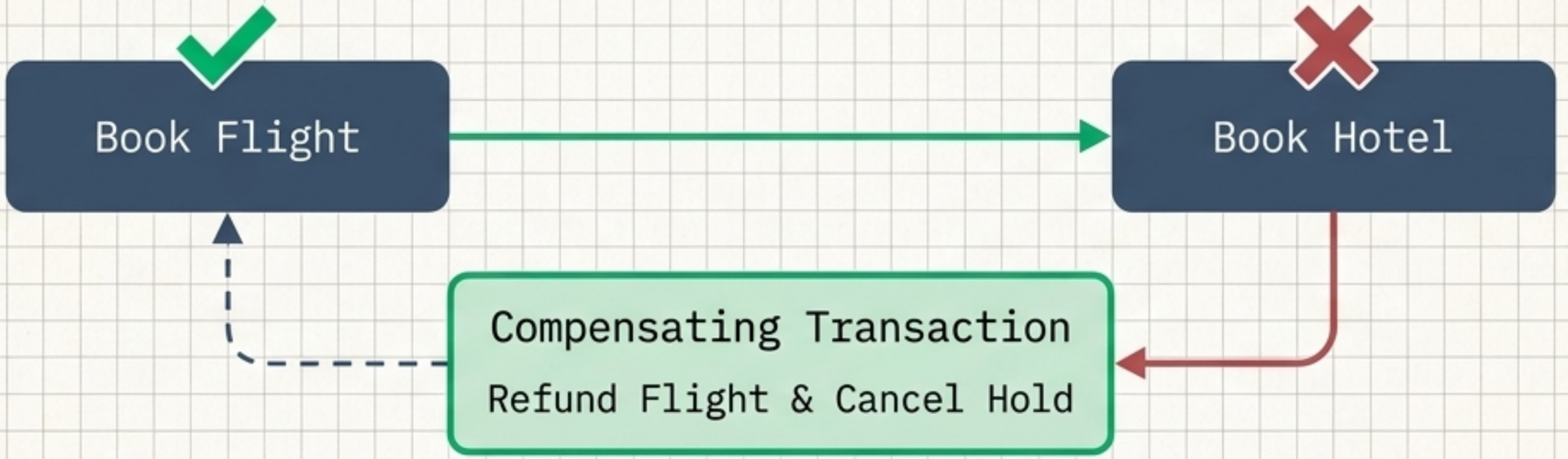
No shared tables between teams. Integrate only via APIs or outbox events.

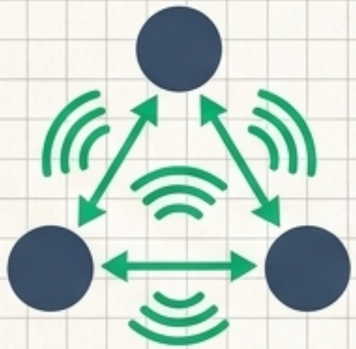
The Cost: Immediate Consistency

You lose cross-table joins. The product team must accept stale reads occasionally to gain independent schema release cadences.

Cross-service business flows require compensating transactions when individual steps fail.

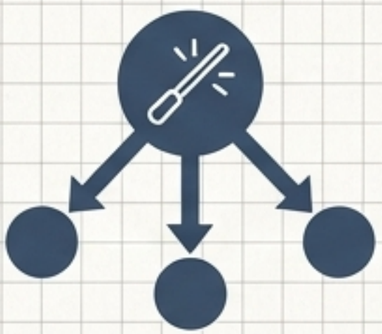
The Saga Pattern





Choreography

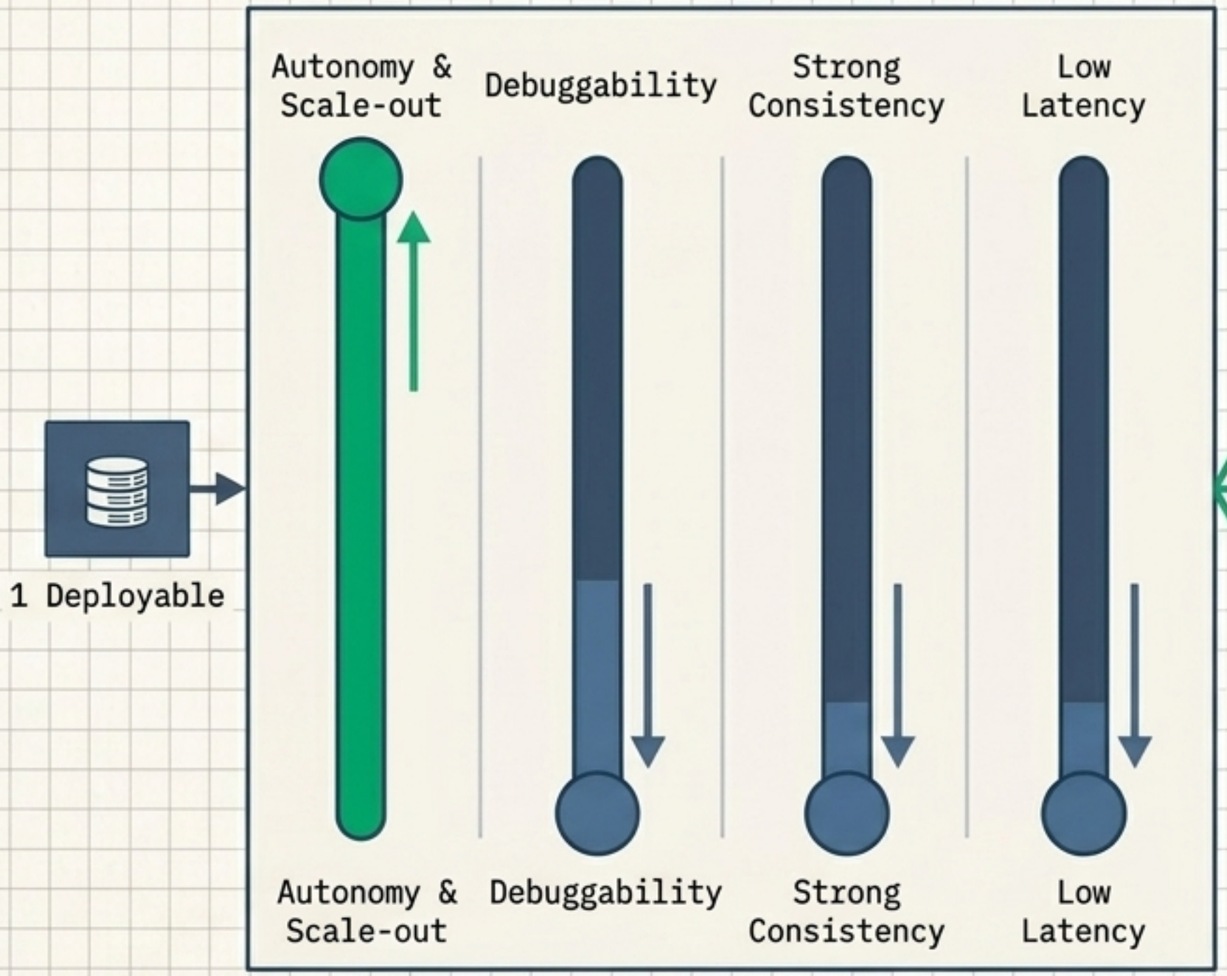
Services react to events without a central brain (scales teams well).

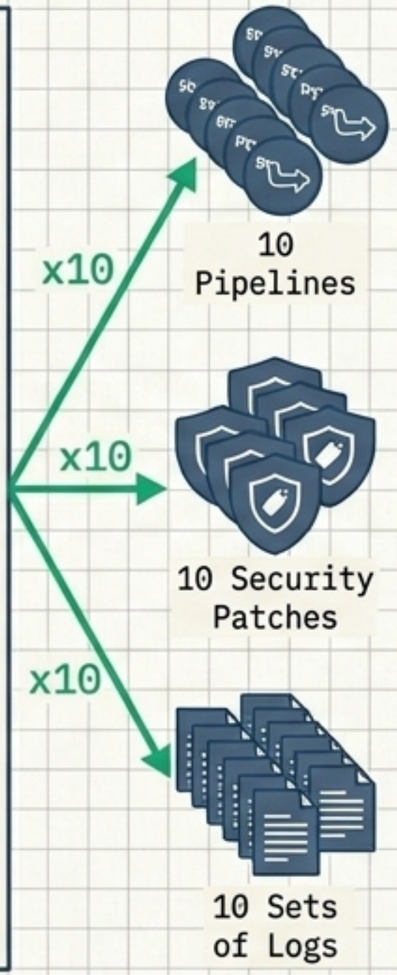


Orchestration

A central workflow engine drives steps (aids visibility for long, human-in-the-loop flows).

Microservices multiply failure surfaces; flexibility is explicitly budgeted against operational simplicity.





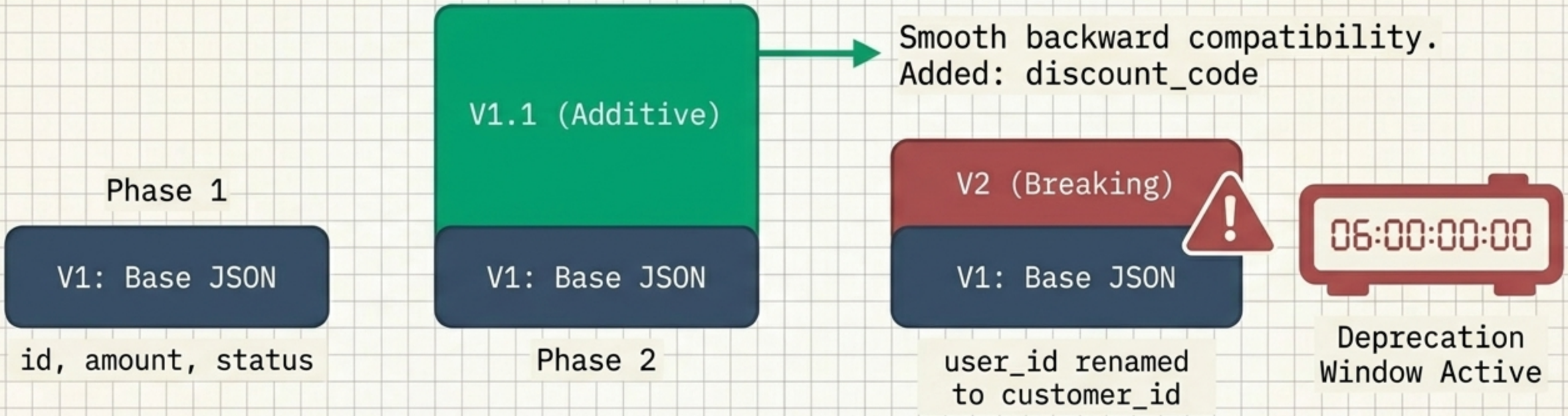
The Flexibility Tax

Fine-grained services improve team parallelism but introduce partial network failures, timeouts, and **idempotency** requirements.

Cost Curves

Teach Product Managers these curves. Finer services mean exponentially higher observability burdens (distributed tracing, complex SLO dashboards).

Independent deployment is a dangerous illusion if API changes break unknown clients.



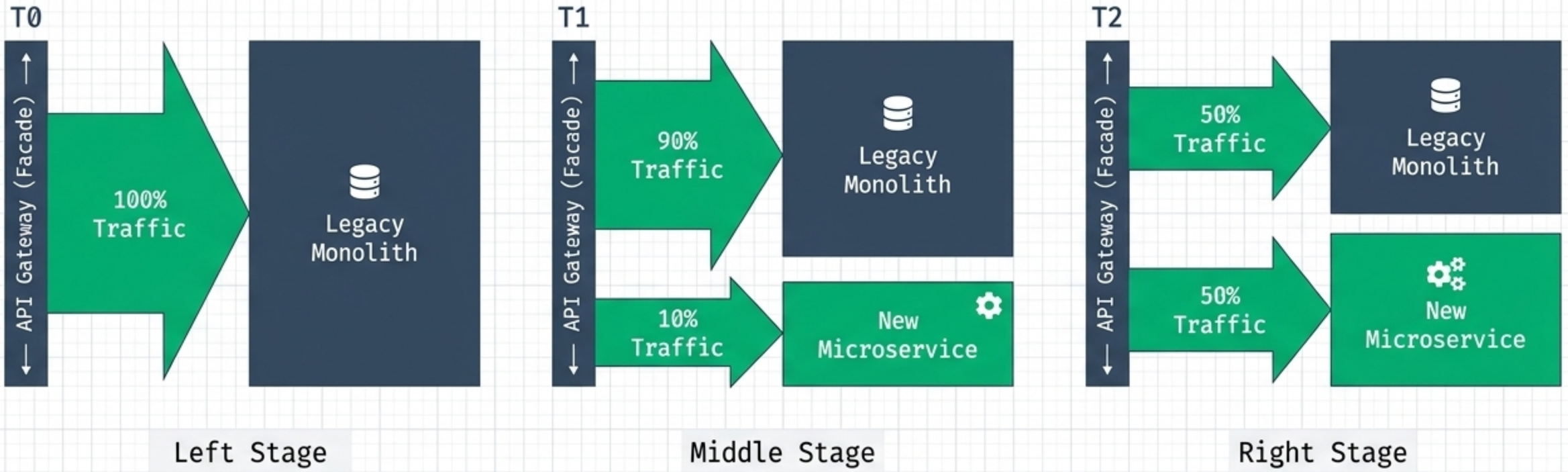
Published Products

Treat public API contracts exactly like published external products. You cannot silently change the rules.

Compatibility Rules

Flexibility requires strict backward compatibility rules, consumer-driven contracts, and documented deprecation windows before removal.

Lower rewrite risk by incrementally routing traffic behind an external facade.



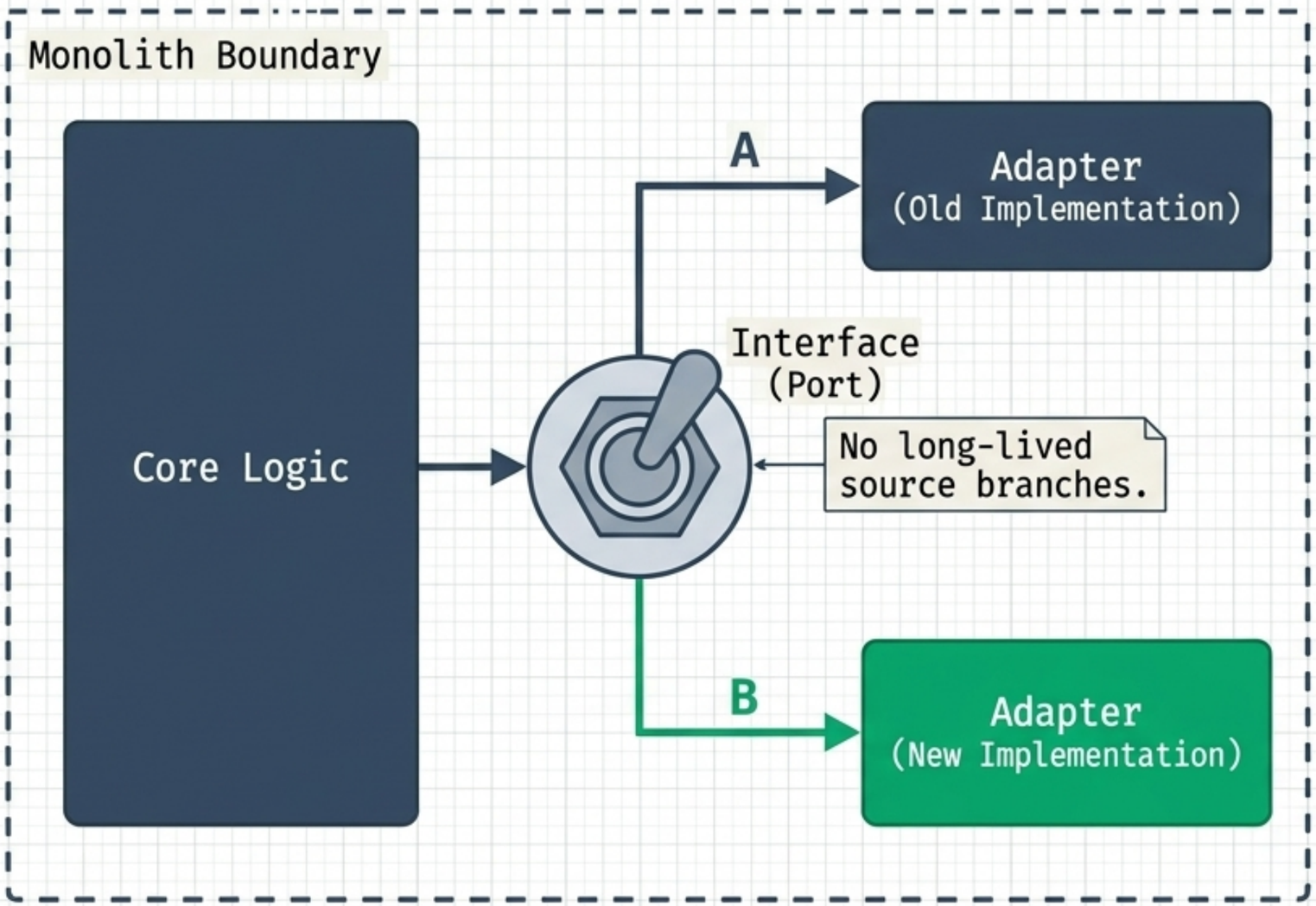
Left Stage

Middle Stage

Right Stage

The Pattern	Why It Works
Incrementally replace a legacy system by routing growing slices of functionality to new implementations over time.	Dual-running traffic slices builds confidence. It entirely avoids the high-risk, long-feedback “stop the world / big-bang” cutover.

Keep your main trunk integrating while hardening new code via internal abstraction boundaries.



Branch by Abstraction

Introduce an abstraction boundary (like a Hexagonal Port) directly into the monolith codebase.

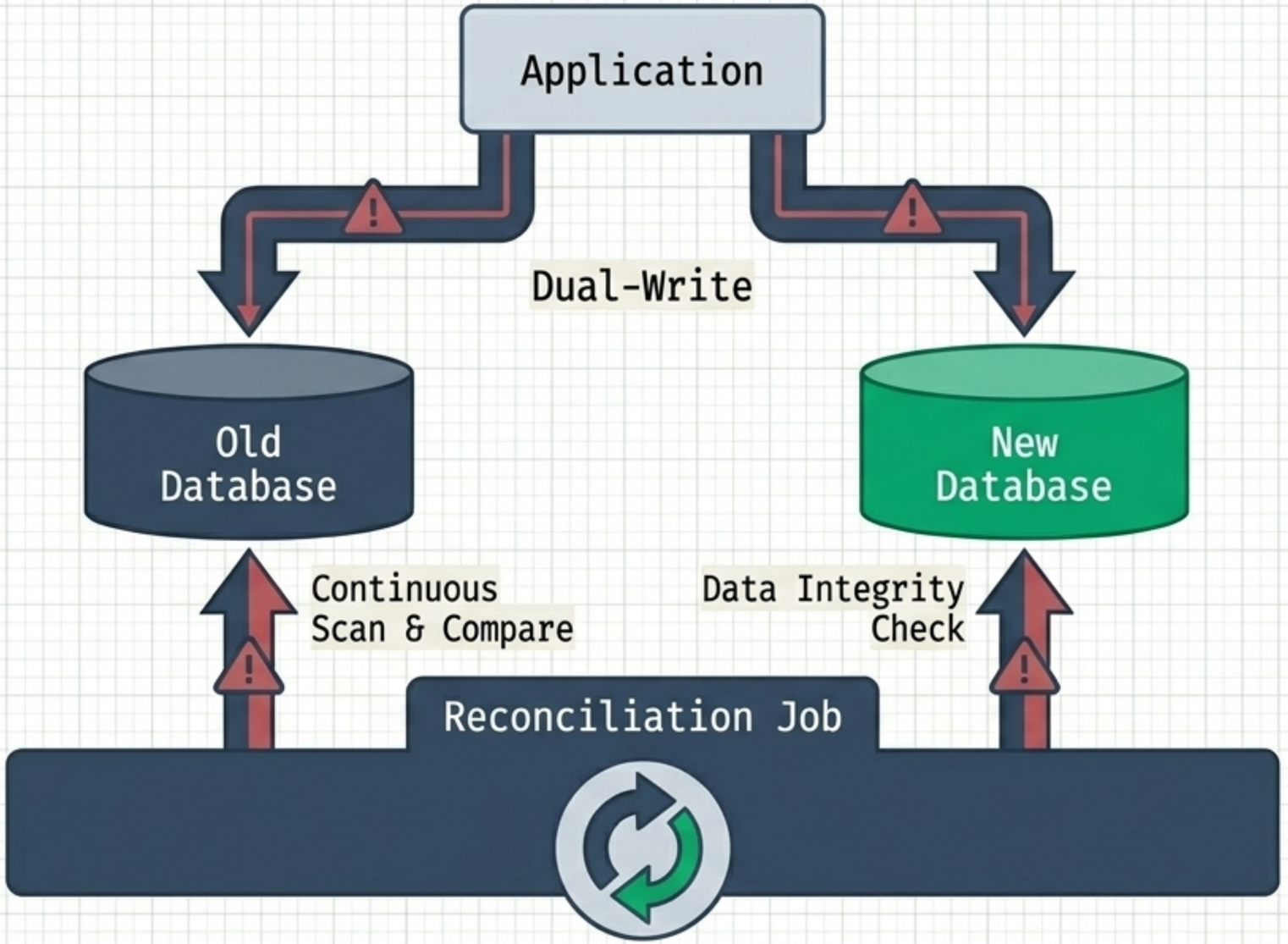
Dependency Direction

Reaching directly into concrete classes blocks extraction. Interfaces allow feature toggling at runtime.

Continuous Integration

Both old and new code live in main. Toggle the new path on in staging while leaving it off in production until hardened.

Flexibility projects fail in the silent data drift between old and new stores.



The Data Dilemma

Migrating code logic is easy; migrating state is hard. The risk of silent corruption is extreme.

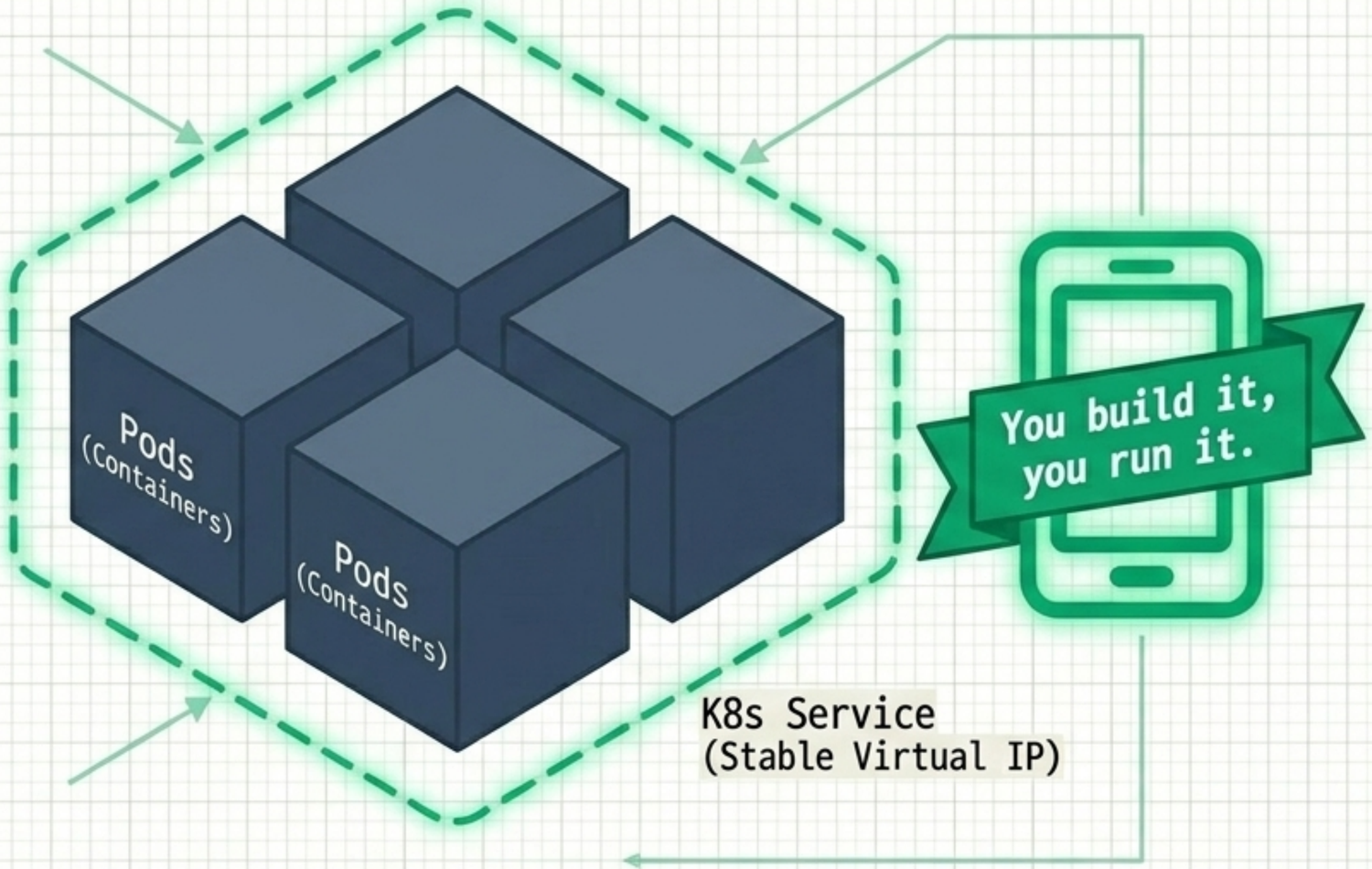
Dual-Writes

For a time, you must write to both stores, verify the outputs via reconciliation, and only flip the read path when mathematically certain.

Product Work

Treat data migration and reconciliation code as first-class product work, not background DDL scripts.

Architectural flexibility requires intense operational maturity and rapid feedback loops.



The Runtime Reality

Primitives like Pods and Services give microservices deployable units and stable network names. But as services multiply, so do secrets and configurations.

DevOps & Ownership

Throwing code ‘over the wall’ to an operations team instantly recreates monolithic coordination via Jira tickets.

The Final Rule

The same team that ships the code carries the pager. Clear SLOs per service are mandatory to survive distribution.